

AICE2005 - Systematic Design

Functional and Non-Functional Requirements

Asieh Salehi Fathabadi

A.Salehi@Soton.ac.uk

University of Southampton

2026-2027

Week Overview

Week 2 | Analysing systems: requirements

Lecture 1: Functional and Non-Functional Requirements

What must the system do, and how well must it do it?

Lecture 2: Gathering and Validating Requirements

How do we find out what stakeholders need, and check we got it right?

Lecture 3: Use Cases and Personas

How do we capture users and their goals as scenarios?

Record your attendance



303258

Lecture 1 Overview

- Requirements, FRs and NFRs
- Quality attributes and measurable NFRs
- Writing precise requirements
- NFRs, architecture and trade-offs
- Practice: two short exercises

Learning Objectives:

- Distinguish and classify functional and non-functional requirements
- Write measurable, testable and unambiguous requirements
- Explain how NFRs shape architecture and force trade-offs
- Begin drafting well-structured requirements for a multi-component system

What is a Requirement?

A requirement is a statement of a capability, quality or constraint that **a system must meet** to satisfy the needs and expectations of its users and other stakeholders.

A requirement can state:

- 1 a service the system should provide
- 2 how it should react to particular inputs
- 3 how it should behave in particular situations

Use the right verb:

shall = mandatory

should = desirable

may = optional

Requirements are the cornerstone of system development: they guide design, implementation and testing.

Functional Requirements (FRs)

Functional requirements describe **what** the system should do: its behaviours, services and functions.

Examples

- The vehicle shall lock its doors automatically when speed exceeds 20 mph.
- The user management module shall allow the creation, update and deletion of user accounts.
- The recommendation engine shall suggest three relevant videos after a student completes a module.

Characteristics

- Observable and testable (pass/fail)
- Describe a behaviour: what the system does for a given input or situation
- Define interactions between the system and its environment

FRs are usually elicited first, because they map directly to what stakeholders want the system to do.

Non-Functional Requirements (NFRs)

Non-functional requirements describe **how well** the system performs its functions: quality attributes and constraints.

Examples

- The system shall respond to user actions within 250 milliseconds (**Performance**).
- The system shall be usable by people with visual impairments (**Accessibility**).
- Data shall be encrypted using AES-256 (**Security**).
- System uptime shall be 99.99% over a rolling 30-day window (**Availability**).
- A sensor node shall run for 12 months on one battery (**Energy**).

A product can meet every functional requirement and still fail: poor usability, high latency or unreliability.

NFRs are harder to express, elicit and validate, which is why they are often the differentiators between competing systems.

Quality requirements say how well; **constraints** limit the design (e.g. "must use the campus network").

Appendix: FR vs. NFR Key Differences

Aspect	Functional (FR)	Non-functional (NFR)
Definition	What the system should do	How well it should do it
Purpose	Features, services, functions	Quality attributes and constraints
Example	"The system shall allow user login"	"The dashboard shall load in under 2 s"
Measurement	Pass/fail on correct output	Benchmarks, metrics, SLAs
Verification	Functional tests (unit, integration)	Performance, load, usability, security tests
Visibility	Directly observable	Often invisible until violated
Impact on design	Drives use cases and logic flow	Shapes architecture and deployment
Scope	Tied to specific functions or modules	Cross-cutting: affects many components
Documented in	Use cases, user stories, functional specs	Quality models, SLAs, system-wide constraints

Let's think and reflect

Exercise: Which is which?



Join at:
vevox.app

ID:
124-998-115



Case study: Smart Study Room

Room sensors, a dashboard showing free rooms, and a study assistant.

Vote on Vevox: FR or NFR?

1. The dashboard shall show each room's temperature.
2. The dashboard shall load within 2 seconds.
3. Staff shall be able to export occupancy as CSV.
4. The dashboard shall work with a screen reader.
5. The system shall log every failed login attempt.
6. A sensor node shall run 12 months on one battery.

Which is which? Discussion

#	Statement	Answer	Why
1	Show each room's temperature	FR	Observable behaviour, pass/fail
2	Dashboard loads within 2 s	NFR performance	A threshold on a function
3	Export occupancy as CSV	FR	A service the system provides
4	Works with a screen reader	NFR accessibility	A quality of use, not a feature
5	Log every failed login	Debatable	FR that serves a security NFR
6	12 months on one battery	NFR constraint	A limit on the hardware design

Rule of thumb: an FR is a behaviour you can check pass/fail; an NFR is a quality of that behaviour with a threshold. Many NFRs are met by adding FRs (logging, encryption, alerts).

Quality Attributes: ISO/IEC 25010

The product quality model (2023 edition) has nine characteristics

Functional suitability

functions meet stated and implied needs

Performance efficiency

speed and resource use under stated conditions

Compatibility

exchanges information and co-exists with other systems

Interaction capability

operated effectively and satisfactorily (was Usability)

Reliability

performs specified functions for a period of time

Security

protects data; access matches authorisation

Maintainability

can be modified by its maintainers

Flexibility

adapts to different or evolving environments (was Portability)

Safety

avoids harm to people, property and environment (new)

Green = the functional characteristic; the rest are what we usually call NFRs. The 2011 edition had eight (Usability, Portability, no Safety); availability sits under Reliability.

From Vague to Measurable

Vague: *"The system shall be fast."*

Fast compared to what? Under what load? For whom? How would we test it?

Measurable: 95% of dashboard page loads shall complete within 2 seconds with 100 concurrent users, measured by a load test on the staging server.

- **Which quality?** dashboard page loads (the attribute)
- **What value and unit?** 2 seconds
- **Under what condition?** 100 concurrent users
- **How often?** 95% of the time (a probability, not "always")
- **How is it measured?** load test on the staging server

Relative words (fast, large, user-friendly) only mean something against a reference: state it.

Five Questions to Remove Ambiguity

Ask	Pin down	Ambiguous words
Which things?	Instances vs classes: which user, every or some?	"the user", "the system"
Which value?	Attribute, unit and reference	"fast", "large", "accurate"
When?	Always, within X s of an event, or in a state	"quickly", "regularly", "after"
How many or how often?	All, at least one, or a percentage	"users", "most", "sometimes"
Under what condition?	The trigger and what should follow	"when needed", "if possible"

Every requirement is a statement about things, values, time, quantity and conditions. Writing it as a logical statement makes any gap visible.

Adapted from ideas in "Introduction to Representing Sentences as Logical Statements" (LessWrong, 2025).

A Requirement Read as Logic

"The recommendation engine shall suggest three relevant videos after a student completes a module."

$\forall s \in \text{Student}, \forall m \in \text{Module}:$

$\text{completes}(s, m, t) \Rightarrow \exists R \subseteq \text{Video}:$

$|R| = 3 \wedge \forall v \in R: \text{relevant}(v, s) \wedge \text{suggested}(s, R, t') \wedge t < t' \leq t + \Delta$

Questions the logic forces us to ask:

- **What does relevant mean?** Define it with a measure, e.g. rated relevant by at least 80% of students.
- **What is Δ ?** "After" needs a time limit: within 5 seconds?
- **Which students?** All students, or only those enrolled? (the quantifier)
- **What if fewer than three videos exist?** A missing case: the requirement is incomplete.

Sentence Templates: EARS

Pattern	Template	Smart Study Room example
Ubiquitous	The <system> shall <response>	The dashboard shall display the room name with every reading.
Event-driven	When <trigger>, the <system> shall <response>	When occupancy exceeds capacity, the dashboard shall mark the room full within 2 s.
State-driven	While <state>, the <system> shall <response>	While a room is occupied, the sensor node shall report every 60 s.
Unwanted behaviour	If <condition>, then the <system> shall <response>	If a sensor node is silent for 5 min, then the dashboard shall show the room as "unknown".
Optional feature	Where <feature>, the <system> shall <response>	Where a noise sensor is fitted, the sensor node shall report sound level.

EARS: Easy Approach to Requirements Syntax (Mavin et al., 2009). "When ... shall ..." is a cause-and-effect statement, the same shape as a logical implication.

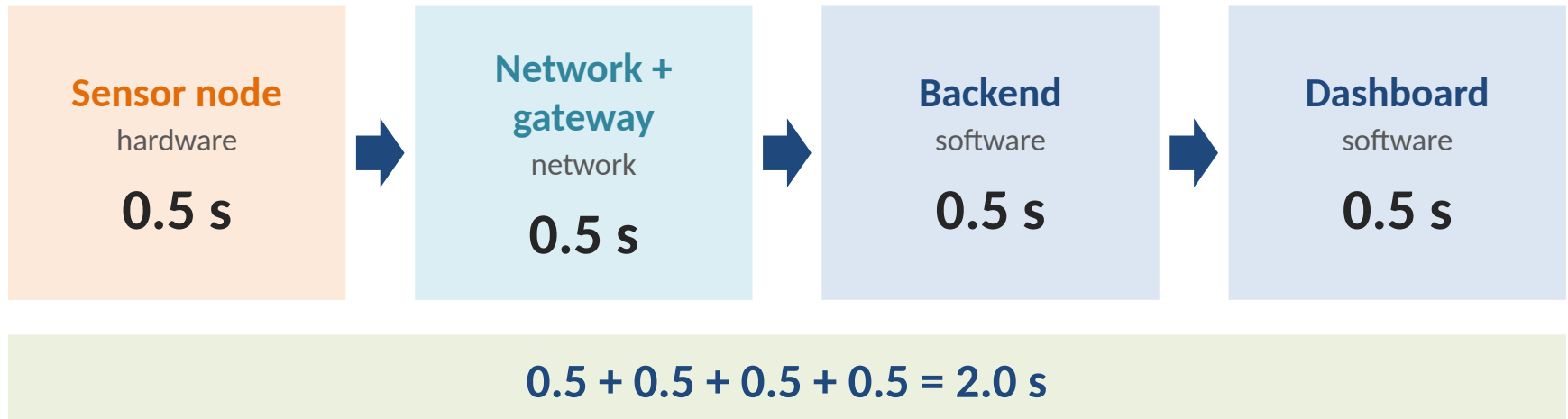
Spot the Problem, Then Rewrite

Weak requirement	Problem	Better
The system should be secure.	Vague, "should" is weak, no test	Passwords shall be stored only as salted Argon2id hashes and never logged.
The interface shall be user-friendly.	Unmeasurable	90% of first-time users shall book a room within 2 minutes without help (test with 10 users).
The system shall be fast, reliable and support many users.	Three requirements in one; no values	Split it: 95% of page loads in 2 s at 100 users; availability of at least 99.5% per month.
The sensor shall be accurate.	No value, unit or reference	Temperature shall be within ± 0.5 °C of a calibrated reference between 10 and 35 °C.

Also watch for: NFRs left out of testing and out of the Agile Definition of Done, and requirements that describe a solution ("use MySQL") instead of a need.

Allocating an NFR Across Components

System NFR: a new sensor reading shall appear on the dashboard within 2 seconds.



- A system-level NFR is split into a budget for each component.
- Each budget becomes a requirement on an interface between components.
- If one component overruns, another must compensate or the requirement is renegotiated.

NFRs Conflict: Managing Trade-offs

Performance vs security

Encryption and authentication add latency and CPU load.

Usability vs security

Stronger authentication adds friction for users.

Accuracy vs battery life

Sampling faster and transmitting more drains the sensor battery.

Personalisation vs privacy

Better recommendations need more personal data.

Resolving a conflict

- Quantify each NFR so the conflict is a number, not an opinion.
- Rank with stakeholders (e.g. MoSCoW: must, should, could, won't).
- Record the decision and the reason.

Summary and What's Next

- **FR vs NFR:** what the system does vs how well it does it
- **Measurable NFRs:** value, unit, condition, proportion, method
- **Precise wording:** five questions, logic, EARS templates
- **Architecture:** NFRs are allocated across components and conflict

Let's read and explore

- ISO/IEC/IEEE 29148:2018. Requirements engineering.
<https://www.iso.org/standard/72089.html>
- ISO/IEC 25010:2023. SQuaRE: Product quality model.
<https://www.iso.org/standard/78176.html>
- Mavin et al. (2009). Easy Approach to Requirements Syntax (EARS). IEEE.
<https://doi.org/10.1109/RE.2009.9>
- Kossiakoff, A. et al. (2011). Systems Engineering Principles and Practice.
<https://www.oreilly.com/library/view/systems-engineering-principles/9780470405482/>

Next Lecture: Gathering and Validating Requirements:
how do we find out what stakeholders really need?